

Practica 2. Aprendizaje en entornos complejos

Salvador de Haro Orenes
salvadorde.haroo@um.es

Miguel Vidal García
miguel.vidalg@um.es

Jose María Belmonte Martínez
josemaria.belmontem@um.es

9 de marzo de 2025

Resumen

En este documento se presenta el trabajo realizado para la segunda parte de la práctica de aprendizaje por refuerzo, utilizando diversos algoritmos para entrenar agentes en diferentes entornos. Se implementarán métodos clásicos como Monte-carlo *on-policy* y *off-policy*, así como algoritmos basados en diferencias temporales como *SARSA* y *Q-Learning*. Además, se explorarán técnicas más avanzadas de control por aproximaciones, como pueden ser: *SARSA Semi-Gradiente* y *Deep Q-Learning*.

Toda la experimentación realizada en el proyecto está disponible en un repositorio de GitHub, conforme a los lineamientos especificaciones establecidas en la tarea.

1. Introducción

El problema abordado en este trabajo se centra en el aprendizaje por refuerzo en entornos complejos mediante el uso de múltiples agentes y diferentes enfoques de aprendizaje. A diferencia de enfoques tradicionales, aquí se estudia la toma de decisiones en un contexto dinámico donde las recompensas pueden no ser inmediatas.

El uso de *Gymnasium* con distintos agentes de aprendizaje por refuerzo, como *Monte-Carlo*, *SARSA* y otros enfoques, permite diseñar agentes capaces de tomar decisiones óptimas en entornos inciertos resulta una tarea más que interesante para aprender sobre estos algoritmos y comprender su implementación y funcionamiento.

Se busca implementar y analizar distintos agentes de aprendizaje por refuerzo, evaluando su desempeño en distintos escenarios. Se pretende comparar la eficacia de los diferentes métodos y estudiar cómo la exploración y explotación afectan el proceso de aprendizaje.

El informe se estructura de la siguiente manera:

- ▶ La primera parte se centra en conocer el entorno *Gymnasium* y realizar unas primeras pruebas para comprobar el funcionamiento de cada uno de los agentes en un entorno sencillo, verificando que realmente funcionan y encuentran una solución.
- ▶ En la segunda parte, para los algoritmos que han demostrado funcionar en un entorno sencillo, se crea un entorno más complejo, *FrozenLake 8x8*, un entorno con 64 estados que añade una mayor dificultad, para analizar su rendimiento y determinar sus limitaciones y capacidades.
- ▶ La última parte estudia algoritmos basados en apro-

ximaciones, como *Deep Q-Learning* y *SARSA Semi-Gradiente*, para entornos continuos (en este caso hemos elegido Mountain Car). Esto evidencia la necesidad de emplear estos métodos, ya que los enfoques tabulares resultan insuficientes debido al tamaño inabordable de la tabla que se generaría.

2. Desarrollo

2.1. Aplicación específica abordada.

Se estudia el aprendizaje por refuerzo en entornos dinámicos mediante diferentes agentes dentro de *Gymnasium*. El objetivo es analizar su capacidad para aprender políticas óptimas en distintos escenarios, desde entornos discretos como *FrozenLake* hasta entornos continuos como MountainCar.

2.2. Describir enfoques existentes en la literatura sobre problemas similares.

El *Reinforcement Learning* (RL) en videojuegos ha avanzado significativamente gracias a plataformas como *Gymnasium*, que proporcionan entornos estandarizados para evaluar algoritmos [1]. Los enfoques principales se dividen en métodos *modelo libre* y *basados en modelo*. Dentro de los primeros, los algoritmos basados en valor, como *Deep Q-Network* (DQN) y sus variantes (Double DQN, Dueling networks, Rainbow), han demostrado un rendimiento superior al humano en juegos Atari mediante la estimación de funciones de valor. Por otro lado, los métodos basados en política, como PPO y actores-críticos (A3C, DDPG), optimizan directamente las políticas de decisión, destacándose en entornos complejos y multi-agente [2].

Más recientemente, los enfoques basados en modelo han cobrado relevancia al aprender representaciones internas del entorno para planificar acciones futuras. Destacan enfoques como *MuZero* [3], que han demostrado alta eficiencia de muestreo y desempeño superhumano en juegos Atari. Las tendencias actuales buscan mejorar la exploración, incrementar la eficiencia de muestreo y favorecer la generalización a nuevos entornos, incorporando arquitecturas avanzadas como transformadores en RL, por ejemplo, *Decision Transformer*, con el objetivo de desarrollar agentes más robustos y eficientes.

2.3. Contexto y antecedentes necesarios para entender el trabajo.

En problemas de toma de decisiones en entornos desconocidos, no se conoce el modelo que rige el entorno, ni las re-

compensas posibles, y mucho menos cuál es la mejor acción a tomar en un estado determinado. Para abordar este desafío, se recurre a la experiencia del agente mientras interactúa con el entorno. Este proceso sigue una secuencia de pasos:

1. El agente observa el estado actual del entorno.
2. Luego, en función de este estado, toma una acción de acuerdo a una política, siempre que no esté en un estado terminal.
3. El entorno cambia a un nuevo estado como resultado de la acción tomada.
4. El agente recibe una recompensa, que se usa para evaluar la acción ejecutada.
5. Finalmente, se repite el proceso para un número específico de episodios, n .

Esta secuencia de pasos varía en función del tipo de agente. Por ejemplo, las técnicas de *Monte-Carlo* esperan a que el agente alcance un estado terminal para considerar el episodio completo y, a partir de ahí, actualizar su política con el objetivo de maximizar la recompensa acumulada a futuro. Por otro lado, las técnicas de diferencias temporales actualizan la política de forma continua, sin necesidad de esperar a un estado terminal.

Por último, cuando el número de estados y acciones es extremadamente grande, se utilizan funciones aproximadas para evaluar estas funciones, lo que da lugar a técnicas más complejas que requieren aproximaciones lineales, como el algoritmo *SARSA Semi-Gradiente*.

2.4. Explicación de los métodos utilizados para abordar el problema.

Se implementan y comparan agentes basados en técnicas tabulares, como *Monte-Carlo*, y agentes que utilizan métodos de control de aproximaciones, como *SARSA Semi-Gradiente* y *Deep Q-Learning*. Inicialmente, los agentes se validan en un entorno simple de Gymnasium para verificar su correcto funcionamiento (métodos tabulares). Luego, se evalúan en el entorno FrozenLake 8x8, que presentan mayor dificultad debido al elevado número de estados.

Finalmente, se analiza su aplicabilidad en entornos continuos, como MountainCar, donde se emplean técnicas de control por aproximaciones (dada su alta capacidad, probarlos en entornos discretos no reflejaría plenamente su potencial, ya que son excesivamente eficientes).

2.5. Justificar cómo el presente trabajo se diferencia o mejora enfoques previos.

A diferencia de estudios previos que se enfocan en un solo tipo de algoritmo o entorno, este trabajo tiene como objetivo comparar múltiples enfoques en distintos escenarios, desde discretos hasta continuos. Además, se analiza la transición de métodos tabulares a métodos con aproximación de valor, resaltando sus ventajas y limitaciones en función del problema abordado.

3. Algoritmos

En esta sección se presentan los distintos algoritmos utilizados en el desarrollo de la práctica, junto con sus principales características. No se profundizará en los detalles de los algoritmos, ya que estos han sido explicados previamente en clase.

3.1. Monte-Carlo on-policy

Este método de aprendizaje por refuerzo basado en la estimación del valor de una política siguiendo la misma política para generar episodios de experiencia. Se utiliza para problemas de control y predicción, actualizando los valores de estado-acción solo al final de cada episodio.

Monte-Carlo on-policy es adecuado para entornos en los que las dinámicas de transición son desconocidas, pero requiere episodios completos para actualizar los valores. En nuestro caso, hemos implementado una política ϵ - *soft* para la selección de la siguiente acción con el método y un decaimiento de ϵ que sigue la siguiente ecuación:

$$\epsilon = \min \left(1, \frac{1000}{n + 1} \right) \quad (1)$$

donde n es el número de episodios transcurridos hasta el momento, lo que permite reducir ϵ gradualmente a medida que el agente avanza en el aprendizaje.

Inicialmente, el agente realiza más exploración, pero con el tiempo, la probabilidad de elegir la mejor acción aumenta a medida que ϵ disminuye.

3.2. Monte-Carlo off-policy

En este caso el método off-policy de *Monte-Carlo* es un enfoque de aprendizaje por refuerzo que se basa en la estimación de la función de valor de las acciones, pero con una diferencia clave: en este caso, el agente aprende de las experiencias generadas por una política diferente de la política que está aprendiendo y evaluando. Es decir, el agente evalúa y mejora una política mientras sigue una política distinta.

Este enfoque permite que el agente pueda aprender de episodios generados por una política más exploratoria (o aleatoria), mientras que evalúa y mejora una política más optimizada, aprovechando así experiencias previas para acelerar el proceso de aprendizaje. Este método es particularmente útil cuando no es posible seguir la política óptima durante el proceso de aprendizaje, pero aún así se desea mejorarla.

El uso de **importance sampling** es crucial para la corrección del valor de las actualizaciones realizadas por la política de comportamiento para reflejar la política objetivo. La fórmula general para realizar el importance sampling:

$$\rho_t = \frac{\pi(a_t|s_t)}{\pi_b(a_t|s_t)} \quad (2)$$

donde:

- ▶ $\pi(a_t|s_t)$ es la probabilidad de tomar la acción a_t en el estado s_t según la política objetivo.
- ▶ $\pi_b(a_t|s_t)$ es la probabilidad de tomar la acción a_t en el estado s_t según la política de comportamiento.

La política $\epsilon - greedy$ y el decaimiento de ϵ son los mismos que los descritos en la sección anterior.

3.3. SARSA

Este algoritmo utiliza una política basada en $\epsilon - greedy$ para la selección de acciones. En lugar de actualizar los valores de las acciones en función de la mejor acción posible a partir de un estado, **SARSA** actualiza la función de valor $Q(s, a)$ basándose en la acción que realmente se tomó en el siguiente paso, lo que lo convierte en un algoritmo **on-policy**.

Este algoritmo sigue una política de exploración-explotación, eligiendo acciones con una probabilidad ϵ aleatoriamente y con una probabilidad $1 - \epsilon$ eligiendo la acción que maximiza el valor $Q(s, a)$. La principal diferencia con otros algoritmos es que **SARSA** actualiza la estimación de los valores de las acciones utilizando la acción realmente tomada en el siguiente estado, en lugar de la acción con el valor más alto posible.

En este algoritmo se introduce un nuevo parámetro α conocido como tamaño de paso. En el contexto de **SARSA**, α controla la velocidad con la que el agente actualiza la estimación de la función de valor $Q(s, a)$ después de cada interacción con el entorno.

En la fórmula de actualización de **SARSA**:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)] \quad (3)$$

α tiene el siguiente efecto:

- ▶ **Cuando α es grande** (cerca de 1), el agente actualiza rápidamente sus estimaciones, lo que acelera el aprendizaje pero puede reducir la estabilidad debido al ruido en las recompensas.
- ▶ **Cuando α es pequeño** (cerca de 0), las actualizaciones son más lentas, lo que suaviza el aprendizaje pero hace que el agente aprenda más despacio.

En resumen, α controla la cantidad de ajuste que se hace a las estimaciones $Q(s, a)$ en cada paso de aprendizaje. Un valor apropiado de α es clave para garantizar un buen balance entre rapidez de aprendizaje y estabilidad.

3.4. Q-Learning

Q-learning es un algoritmo de aprendizaje por refuerzo **off-policy**. La fórmula de actualización de **Q-learning** es la siguiente:

$$Q(S, A) \leftarrow Q(S, A) + \alpha [r + \gamma \max_a Q(S', a) - Q(S, A)] \quad (4)$$

donde, $\max_a Q(S', a)$ es el valor máximo de las acciones posibles en el siguiente estado s' , indicando que **Q-learning**

utiliza la mejor acción disponible para actualizar el valor de $Q(s, a)$.

Q-learning es un algoritmo **off-policy**, lo que significa que actualiza los valores utilizando la mejor acción disponible, sin importar la política seguida por el agente. Con una adecuada política de exploración, como $\epsilon - greedy$, y suficiente exploración de los estados, *Q-learning* puede converger a la política óptima. Además, el agente equilibra la exploración de nuevas acciones con la explotación de las mejores acciones conocidas, lo que le permite aprender eficazmente sobre el entorno.

3.5. SARSA Semi-Gradiente

El algoritmo **SARSA Semi-Gradiente** es una versión de **SARSA** que utiliza aproximación de función para representar los valores $Q(s, a)$ mediante un vector de pesos w . En lugar de actualizar los valores de acción directamente, como en el caso de **SARSA** tradicional, **SARSA Semi-Gradiente** ajusta los pesos w de la función de aproximación utilizando gradientes, lo que permite un enfoque más flexible para manejar espacios de estado grandes.

La actualización de los pesos w en **SARSA Semi-Gradiente** se realiza de acuerdo con la fórmula:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w}) \quad (5)$$

donde:

- ▶ En lugar de almacenar valores $Q(S, A)$ en una tabla, se usa una función parametrizada con pesos $\mathbf{w}$, lo que permite manejar espacios de estado grandes o continuos.
- ▶ Diferencia temporal (TD Error) el término:

$$R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w}) \quad (6)$$

representa la diferencia entre la recompensa observada y la estimación actual, lo que indica cuánto debe corregirse la función de valor.

- ▶ La actualización ajusta los pesos en la dirección del gradiente para minimizar error.

En este caso, w representa los **pesos de una función de aproximación** (como una red neuronal o una tabla con valores de función aproximados). Esto permite manejar espacios de estado más grandes o continuos, donde no es factible almacenar explícitamente todas las estimaciones de Q .

3.5.1. Tile Coding

Tile Coding es una técnica utilizada para representar el espacio de estados de manera eficiente en problemas de aprendizaje por refuerzo, especialmente cuando los espacios de estado son continuos o de alta dimensión. En lugar de utilizar una representación directa del espacio de estados, Tile Coding divide el espacio en varias rejillas o "tiles", donde cada rejilla tiene un conjunto de bits que indican si un estado particular pertenece a la rejilla.

El Tile Coding divide el espacio de estados en varias rejillas (tiles), donde cada rejilla cubre una parte del espacio. Cada

estado se representa por un vector binario: si el estado pertenece a una rejilla, se activa un bit en ese vector. Finalmente, la representación del estado se forma sumando los bits activados por todas las rejillas a las que pertenece el estado. Esto permite representar el estado de forma compacta y eficiente.

Este enfoque permite representar espacios de estados grandes de manera eficiente y mejora la capacidad de generalización de los algoritmos de aprendizaje.

3.6. Deep Q-Learning

Deep Q-Learning (DQN) extiende el Q-Learning utilizando redes neuronales para aproximar la función de valor $Q(S, A)$, permitiendo manejar espacios de estado grandes. La red neuronal $Q(S, A; w)$ estima los valores de acción y se actualiza minimizando el error de la diferencia temporal (TD error):

$$\text{TD Target} = R + \gamma \max_{A'} Q(S', A'; w^-) \quad (7)$$

Para mejorar la estabilidad, se emplea una **red de destino** $Q(S, A; \theta^-)$, una copia congelada de la red principal, y una **memoria de experiencia**, donde se almacenan y muestrean transiciones (S, A, R, S') de manera aleatoria. DQN ha permitido avances en aprendizaje por refuerzo profundo, logrando resultados sobresalientes en videojuegos y entornos complejos.

4. Evaluación/Experimentos

En esta sección se detallan los experimentos realizados y los métodos utilizados para su evaluación.

4.1. Configuración Experimental

Para evaluar el rendimiento del algoritmo, se utilizaron diversos entornos de **Gymnasium**, una biblioteca ampliamente utilizada en aprendizaje por refuerzo. Los entornos seleccionados fueron:

- **FrozenLake 4×4 y FrozenLake 8×8:** Son versiones del mismo problema con tamaños de tablero de 4×4 y 8×8, respectivamente. El agente debe moverse por una cuadrícula helada hasta alcanzar un objetivo, evitando caer en agujeros. En la versión utilizada, el entorno es **determinista**, lo que significa que cada acción del agente siempre produce el mismo resultado esperado.
- **MountainCar:** En este entorno, un automóvil debe ascender una colina utilizando únicamente la aceleración hacia adelante o hacia atrás. Debido a la gravedad, el coche no puede alcanzar la cima en un solo intento, por lo que debe balancearse hacia adelante y atrás para generar el impulso suficiente.

Estos entornos permiten evaluar el desempeño del agente en situaciones con diferentes niveles de dificultad y requerimientos de planificación.

4.2. Métricas utilizadas para evaluar el desempeño del método propuesto

Para evaluar el desempeño de los métodos propuestos, se han utilizado varios tipos de métricas.

- Se presentan gráficos que muestran el beneficio promedio a medida que avanzan los episodios, lo que permite observar la evolución del desempeño del agente durante el entrenamiento
- Se incluye el análisis de la longitud de los episodios a lo largo del proceso de aprendizaje:

$$f(t) = \frac{\sum_{i=1}^t R_i}{t} \quad \text{para } t = 1, 2, \dots, \text{NumeroEpisodios} \quad (8)$$

Además, para los algoritmos tabulares, se presenta también la tabla Q, que proporciona información detallada sobre los valores de acción para cada estado, permitiendo visualizar cómo se ajustan las políticas a medida que el agente mejora su desempeño. Finalmente, se incluye un GIF que ilustra el camino seguido por el agente una vez entrenado, permitiendo observar su comportamiento en el entorno.

4.3. Resultados obtenidos

En esta sección se presentan los resultados obtenidos durante la evaluación de los algoritmos propuestos. Tanto en el entorno MountainCar como en FrozenLake.

4.3.1. FrozenLake 4x4

Los resultados obtenidos han sido para un entrenamiento de 10,000 episodios para cada algoritmo, para el entorno FrozenLake 4x4 por defecto sin resbalamiento.

Tabla 1: Resultados obtenidos

	<i>MC on-policy</i>	<i>MC off-policy</i>	<i>SARSA</i>	<i>Q-Learning</i>
rec ¹	0.6479	0.649	0.5903	0.6452
len ²	10	10	100	10

¹ rec = recompensa promedio

² len = longitud último episodio

4.3.2. FrozenLake 8x8

Los resultados obtenidos han sido para un entrenamiento de 10,000 episodios para cada algoritmo, para el entorno FrozenLake 8x8 por defecto sin resbalamiento.

Tabla 2: Resultados obtenidos

	<i>MC on-policy</i>	<i>MC off-policy</i>	<i>SARSA</i>	<i>Q-Learning</i>
rec ¹	0.712	0.003	0.6781	0.6575
len ²	20	100	100	20

¹ rec = recompensa promedio

² len = longitud último episodio

4.3.3. MountainCar

Los resultados obtenidos han sido para un entrenamiento de 10,000 episodios en el algoritmo *SARSA SemiGradiente* y de

5000 episodios para el *Deep Q-Learning* para cada algoritmo, para el entorno FrozenLake 8x8 por defecto sin resbalamiento.

Tabla 3: Resultados obtenidos

	<i>SARSA</i>	<i>SemiGradiente</i>	<i>Deep Q-Learning</i>
rec ¹	-128.739		-196.2926
len ²	180		185

¹ rec = recompensa promedio

² len = longitud último episodio

4.4. Análisis crítico de los resultados y comparación con otros enfoques.

Para un análisis más detallado, dividiremos la sección en varios apartados, siguiendo la estructura del notebook. Para una explicación más profunda y exhaustiva, se puede consultar el notebook, donde todo está explicado de manera más completa y detallada https://github.com/salvadeharo10/RL_DHBV.

4.4.1. Para el entorno FrozenLake 4x4

A continuación, se realiza un análisis de los resultados presentados en la tabla 1. En ella, se muestran dos métricas clave para los algoritmos de aprendizaje por refuerzo: la recompensa promedio y la longitud del último episodio. Los resultados se presentan para cuatro algoritmos: Monte Carlo on-policy, Monte Carlo off-policy, SARSA y Q-Learning.

- ▶ MC on-policy obtiene una recompensa promedio de **0.6479**, lo que sugiere un buen rendimiento en términos de alcanzar el objetivo. Este valor es relativamente alto, lo que indica que el agente logró recompensas positivas con frecuencia durante el entrenamiento.
- ▶ MC off-policy presenta un valor ligeramente superior de **0.649**, lo que indica un rendimiento similar al de MC on-policy, pero con una ligera mejora. Esto podría sugerir que el uso de una estrategia off-policy permitió una exploración más efectiva.
- ▶ SARSA muestra una recompensa promedio de **0.5903**, que es más baja en comparación con los métodos anteriores. Este resultado puede indicar que SARSA, al ser un algoritmo basado en políticas, ha tenido más dificultades para explorar y aprender una política óptima en este entorno.
- ▶ Q-Learning, con un valor de **0.6452**, tiene un rendimiento cercano al de MC on-policy y off-policy, lo que sugiere que Q-Learning ha sido igualmente efectivo en términos de alcanzar el objetivo.

Los algoritmos **Monte Carlo on-policy**, **Monte Carlo off-policy** y **Q-Learning** lograron un rendimiento similar con recompensas promedio altas y longitudes de episodio cortas, lo que indica que estos métodos fueron más eficientes en aprender la política óptima. Por otro lado, **SARSA** presentó una recompensa promedio más baja y una longitud de episodio significativamente más larga, lo que sugiere que tuvo un rendimiento inferior en este entorno, posiblemente debido a su naturaleza basada en políticas que podría haber dificultado la convergencia.

4.4.2. Para el entorno FrozenLake 8x8

A continuación, se realiza un análisis de los resultados obtenidos en la tabla 2:

- ▶ MC on-policy muestra una recompensa promedio de **0.712**, lo que indica un buen desempeño en términos de alcanzar el objetivo de manera eficiente. Además, la longitud del último episodio es de **20**, lo que sugiere que el agente fue capaz de completar los episodios rápidamente y de forma efectiva (esto se puede ver en la evolución de los episodios en el notebook).
- ▶ MC off-policy, por otro lado, presenta una recompensa promedio de **0.003**, lo cual es extremadamente bajo, lo que sugiere que este algoritmo no logró obtener recompensas positivas de manera consistente. La longitud del último episodio de **100** refleja una mayor dificultad para el agente en aprender una política eficiente en este entorno. Esto se debe a que el agente cae repetidamente en hoyos o alcanza el número máximo de pasos sin encontrar la solución. Como se muestra en el notebook, su recompensa disminuye a medida que avanzan los episodios, ya que, aunque en raras ocasiones llega a la meta, esto ocurre con muy poca frecuencia.
- ▶ SARSA muestra una recompensa promedio de **0.6781**, lo que indica un buen desempeño, aunque no tan alto como el de Monte Carlo on-policy. La longitud del último episodio de **100** refleja una mayor dificultad para alcanzar el objetivo, lo cual puede ser un indicativo de una convergencia más lenta debido a la naturaleza basada en políticas de SARSA.
- ▶ Q-Learning, con una recompensa promedio de **0.6575**, muestra un rendimiento competitivo. Su longitud de episodio de **20** sugiere que ha sido bastante eficiente en alcanzar el objetivo, similar a Monte Carlo on-policy.

En resumen, *Monte Carlo on-policy* ha sido el algoritmo más efectivo en términos de recompensa promedio, mientras que *Q-Learning* se destacó en cuanto a la longitud del episodio. Por otro lado, *Monte Carlo off-policy* presentó un rendimiento pobre. Podemos decir que en esta prueba el más destacado ha sido *Q-Learning* por su eficiencia en la longitud del episodio.

4.4.3. Para el entorno Mountain Car

Por último, en cuanto a los resultados obtenidos con *SARSA* *SemiGradiente* y *Deep Q-Learning*, se observa lo siguiente:

- ▶ SARSA *SemiGradiente* muestra una recompensa promedio de **-128.739**, lo que indica un rendimiento negativo en comparación con los otros algoritmos evaluados previamente. La longitud del último episodio es de **180**, lo que sugiere que, aunque el agente pudo completar episodios, tuvo dificultades para aprender una política eficaz.
- ▶ Deep Q-Learning, con una recompensa promedio de **-196.293**, presenta un desempeño aún más bajo que SARSA *SemiGradiente*, lo que indica que este algoritmo tuvo un peor rendimiento en términos de alcanzar el objetivo. La longitud del último episodio de **185** también refleja una mayor dificultad en su proceso de aprendizaje, ya que el agente tardó más en completar los episodios.

En resumen, tanto *SARSA SemiGradiente* como *Deep Q-*

Learning tuvieron un rendimiento deficiente en términos de recompensa promedio ambos encontrando la solución pero con Deep Q-Learning mostrando el peor desempeño en ambas métricas además de un tiempo de ejecución bastante más elevado (y una longitud de episodio de casi máxima todo el rato).

5. Conclusiones

En este estudio se ha demostrado que los algoritmos de *Monte Carlo on-policy* y *Q-Learning* son los más efectivos en términos de recompensa promedio. Además, *Q-Learning* se destacó también por su eficiencia en la longitud del episodio. Por otro lado, *Monte Carlo off-policy* presentó un rendimiento pobre, con una recompensa promedio extremadamente baja y una mayor longitud de episodio, lo que indica que la política fuera de línea no fue efectiva en este entorno.

SARSA y *Q-Learning* también mostraron buenos resultados, con Q-Learning logrando mayor eficiencia en términos de la longitud de los episodios. Sin embargo, algunos de los resultados pueden haberse visto afectados por la configuración inicial de los episodios y la falta de exploración en otros entornos.

Las limitaciones de este estudio incluyen el análisis limitado a un entorno específico y la falta de experimentación con diferentes configuraciones de hiperparámetros, lo cual podría haber influido en los resultados obtenidos. En futuras investigaciones, sería interesante explorar otros entornos más complejos o realizar una comparación más exhaustiva entre diferentes técnicas de optimización de algoritmos.

Este trabajo subraya la importancia de comparar algoritmos de aprendizaje por refuerzo y su impacto potencial en el campo, al proporcionar una comprensión más clara de las ventajas y desventajas de distintas aproximaciones. Las líneas futuras de estudio podrían enfocarse en la implementación de estos algoritmos en escenarios más desafiantes, como el control de robots o en la aplicación de métodos híbridos que combinen enfoques basados en políticas y valores.

Referencias

- [1] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). *OpenAI Gym*. arXiv preprint arXiv:1606.01540.
- [2] Xiao, Z. (2024). *AC: Actor-Critic*. En *Reinforcement Learning: Theory and Python Implementation* (pp. 237–287). Springer.
- [3] Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., et al. (2020). *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. *Nature*, 588(7839), 604–609.